

```

...     print(key, value)
...
b 2
c 3
a 1

```

- An `OrderedDict` provides a method, `pop_item()` to pop an element from either end of the dictionary. This method accepts a parameter `last`, that decides which end of the dictionary we wish to pop. If `last = False`, elements are returned in first in first out [FIFO] order, else they are returned in last in first out [LIFO] order. If no parameter is supplied, the parameter defaults to `True`. Calling `popitem()` on an empty dictionary will raise a `KeyError`.

```

>>> od = OrderedDict({'a': 1, 'b': 2, 'c': 3})
>>> od.popitem() #if no parameter supplied, last = True
('c', 3)
>>> od = OrderedDict({'a': 1, 'b': 2, 'c': 3})
>>> od.popitem(last = False)
('a', 1)
>>> od = OrderedDict()
>>> od.popitem()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'dictionary is empty'

```

- If we wish to move keys to either of the corners of an ordered dictionary, we need not delete the key and reinsert. Rather, we are provided with a method `move_to_end(key, last=True, False)`, where `key` is the element which we wish to delete. The element is moved to either the right or the left end of the dictionary depending on `last`, whether it is `True` or `False` respectively. If the dictionary does not contain the element supplied by the `key` param, we will encounter a `KeyError`.

```

>>> od = OrderedDict({'a': 1, 'b': 2, 'c': 3})
>>> od.move_to_end('a'); #last defaults to True
>>> od
OrderedDict([('b', 2), ('c', 3), ('a', 1)])

```

The above example would otherwise require two operations, first operation to delete the element 'a', second operation to reinsert the element in the dictionary.